



Projektarbeit

Internationale Hochschule Duales Studium

Studiengang: Informatik

Fallstudie Software-Engineering: Dokumentation Lebens-Finanzplaner, Konzeption und Umsetzung einer webbasierten Anwendung zur persönlichen Finanzplanung

Noah Roosen, Grit Unzner, Justas Vaicekauskas, Asal Ghabeljoo, Thomas Relovsky

Betreuende Person: Jens Gulden

Abgabedatum: 24.07.2026

Inhaltsverzeichnis

Inhaltsverzeichnis	II
1 Kurzfassung	1
2 Einleitung	1
2.1 Ausgangssituation und Motivation	1
2.2 Zielsetzung	2
2.3 Abgrenzung	2
2.4 Aufbau der Dokumentation	2
3 Projektziele und Anforderungen	2
3.1 Zielgruppe	3
3.2 Funktionale Anforderungen	3
3.3 Nichtfunktionale Anforderungen	4
3.4 Anwendungsfälle	4
4 Technologische Grundlage	4
4.1 React und Vite im Frontend	4
4.2 Flask im Backend	5
4.3 PostgreSQL und Supabase	5
4.4 Docker und Docker Compose	5
4.5 Git und GitHub	5
5 Systemarchitektur	6
5.1 Architekturübersicht	6
5.2 Frontend-Schicht	6
5.3 Backend-Schicht	6
5.4 Kommunikation und CORS	6
6 Datenbankkonzept und Migration	6
6.1 Entwicklung der Datenbankstrategie	7
6.2 Relationales Zielmodell	7

6.3	Überführung von JSON in Tabellen.....	7
6.4	Vorteile und Risiken der Migration	7
6.5	Datenschutz und Datenminimierung	8
7	Implementierung zentraler Funktionen	8
7.1	Registrierung	8
7.2	Login und Sitzung	8
7.3	Finanzprofil.....	9
7.4	Ziele und Lebensereignisse	9
7.5	Passwort vergessen Funktion	9
7.6	E-Mail-Service	9
7.7	FAQ-Seite.....	10
7.8	Prognosen und Finanzassistent.....	10
8	Sicherheitskonzept	10
8.1	Passwortspeicherung	10
8.2	Sitzungs- und Cookie-Konfiguration.....	10
8.3	Schutz des Passwort-Reset-Prozesses	11
8.4	Eingabevalidierung und Fehlerbehandlung.....	11
8.5	Grenzen des aktuellen Sicherheitsstands	11
9	Bereitstellung und Inbetriebnahme	11
9.1	Voraussetzungen	11
9.2	Start mit Docker	12
9.3	Start ohne Docker	12
10	Tests und Qualitätssicherung	12
10.1	Teststrategie	12
10.2	Manuelle Tests	12
10.3	Automatisierte Tests	13
10.4	Qualitätskriterien	13
10.5	Festgestellte Grenzen	13
11	Projektorganisation und Herausforderungen	13

11.1	Aufgabenverteilung	14
11.2	Wechsel der Datenbanktechnologie.....	14
11.3	Schnittstellenabstimmung	14
11.4	Versionsverwaltung	14
11.5	Erkenntnisse aus dem Projekt	15
12	Bewertung und Ausblick.....	15
12.1	Bewertung der Zielerreichung	15
12.2	Technische Weiterentwicklungen	15
12.3	Fachliche Weiterentwicklungen.....	15
12.4	Risiken bei einer Produktivsetzung	16
13	Fazit	16

1 **Kurzfassung**

Der Lebens-Finanzplaner ist eine webbasierte Anwendung, die Nutzerinnen und Nutzer bei der strukturierten Erfassung, Auswertung und Planung ihrer persönlichen Finanzsituation unterstützt. Im Mittelpunkt stehen die Registrierung und Anmeldung, die Speicherung eines individuellen Finanzprofils, die Verwaltung finanzieller Ziele und Lebensereignisse sowie die Bereitstellung von Prognosen und unterstützenden Informationsangeboten. Die Anwendung folgt einer Client-Server-Architektur. Das Frontend wurde mit React umgesetzt, während im Backend das Python-Framework Flask verwendet wird. Als persistente Datenhaltung dient eine relationale PostgreSQL-Datenbank, die über Supabase bereitgestellt wird.

Eine zentrale technische Herausforderung des Projekts war die mehrfach angepasste Datenbankstrategie. Frühere Entwicklungsstände waren zunächst auf Azure Cosmos DB und damit auf eine dokumentenorientierte beziehungsweise JSON-basierte Speicherung ausgerichtet. Im weiteren Projektverlauf erfolgte der Wechsel zu Supabase und PostgreSQL. Dadurch mussten die bestehenden Datenmodelle, Zugriffsservices und Routen an ein relationales Datenbankschema angepasst werden. Insbesondere verschachtelte Daten wie finanzielle Ziele und Lebensereignisse wurden aus JSON-Strukturen in eigene Tabellen überführt. Ein Migrationsskript unterstützt die Übernahme bereits vorhandener Datensätze.

Die Dokumentation beschreibt Anforderungen, Architektur, Datenbankkonzept, zentrale Implementierungsberichte, Sicherheitsmaßnahmen, Bereitstellung, Qualitätssicherung und organisatorische Erkenntnisse. Darüber hinaus werden die erreichten Ergebnisse kritisch bewertet und mögliche Weiterentwicklungen aufgezeigt.

2 **Einleitung**

2.1 **Ausgangssituation und Motivation**

Private Finanzplanung ist häufig über mehrere Konten, Verträge, Tabellen und Notizen verteilt. Dadurch fehlt vielen Personen eine zentrale Sicht auf Einnahmen, Ausgaben, Vermögen, Verbindlichkeiten und längerfristige Ziele. Besonders bei größeren Lebensentscheidungen, beispielsweise einem Umzug, einer Immobilie, einer Familiengründung oder einem früheren Renteneintritt, reicht eine isolierte Betrachtung einzelner Monatswerte nicht aus. Eine digitale Anwendung kann finanzielle Informationen bündeln, Szenarien nachvollziehbar darstellen und damit die Entscheidungsgrundlage verbessern.

Das Projekt Lebens-Finanzplaner greift diese Problemstellung auf. Ziel war nicht die Entwicklung einer vollständigen Bank- oder Anlageberatung, sondern einer übersichtlichen Webanwendung zur persönlichen Planung. Nutzerinnen und Nutzer sollen ihre Ausgangslage erfassen, finanzielle Ziele hinterlegen und ausgewählte Entwicklungen nachvollziehen können. Ergänzend bietet die

Anwendung Informations- und Unterstützungsfunktionen, unter anderem eine FAQ-Seite und einen eingebundenen Finanzassistenten.

2.2 Zielsetzung

Das übergeordnete Projektziel besteht in der Entwicklung eines lauffähigen Prototyps, der die wichtigsten Schritte einer persönlichen Finanzplanung technisch abbildet. Der Prototyp soll eine verständliche Benutzeroberfläche, ein modular aufgebautes Backend und eine dauerhafte nutzbare Datenhaltung kombinieren. Ein besonderer Schwerpunkt liegt auf der konsistenten Speicherung nutzerspezifischer Daten auf einer sicheren Verarbeitung von Zugangsdaten.

- Registrierung neuer Nutzerkonten sowie An- und Abmeldung
- Erfassung und Speicherung eines persönlichen Finanzprofils
- Verwaltung von finanziellen Zielen und Lebensereignissen
- Bereitstellung von Prognose- und Szenario Funktionen
- Passwort-Wiederherstellung über einen zeitlich begrenzten E-Mail-Link
- Bereitstellung einer FAQ-Seite und unterstützender Inhalte
- Reproduzierbarer Start der Anwendung mit Docker

2.3 Abgrenzung

Der Lebens-Finanzplaner stellt keine rechtsverbindliche Finanz-, Steuer- oder Anlageberatung dar. Prognosen beruhen auf eingegebenen Werten und vereinfachen Annahmen. Reale Marktentwicklungen, individuelle Steuerbedingungen, Inflation oder unerwartete Lebensereignisse können nur begrenzt berücksichtigt werden. Die Anwendung ist daher als Planungs- und Lernwerkzeug zu verstehen. Ebenfalls nicht Gegenstand des Prototyps sind direkte Bankanbindungen, Zahlungsfunktionen, automatisierte Wertpapiertransaktionen oder eine produktive Identitätsprüfung.

2.4 Aufbau der Dokumentation

Nach der Darstellung von Anforderungen und technischen Grundlagen folgt die Beschreibung der Systemarchitektur. Anschließend werden das Datenbankkonzept und die Migration von dokumentenorientierten JSON-Strukturen zu relationalen Tabellen erläutert. Darauf aufbauend beschreibt die Dokumentation zentrale Funktionen, Sicherheitsmaßnahmen, Inbetriebnahme und Tests. Abschließend werden Projektorganisation, Herausforderungen, Weiterentwicklungsmöglichkeiten und das Projektergebnis bewertet.

3 Projektziele und Anforderungen

3.1 Zielgruppe

Die Anwendung richtet sich an Privatpersonen, die ihre finanzielle Situation strukturiert erfassen und längerfristige Entscheidungen vorbereiten möchten. Die Zielgruppe benötigt keine vertieften Kenntnisse in Buchhaltung oder Finanzmathematik. Aus diesem Grund muss die Bedienung verständlich, visuell klar und auf wenige zentrale Eingaben reduziert sein. Gleichzeitig sollen die Daten so strukturiert gespeichert werden, dass spätere Auswertungen und Erweiterungen möglich bleiben.

3.2 Funktionale Anforderungen

ID	Anforderungen	Beschreibung
F01	Registrierung	Ein Konto wird mit Vornamen, Nachname, Geburtsdatum, E-Mail-Adresse und Passwort angelegt.
F02	Anmeldung	Registrierte Personen können sich mit E-Mail-Adresse und Passwort anmelden.
F03	Abmeldung	Eine bestehende Sitzung kann beendet werden.
F04	Finanzprofil	Eingaben zur finanziellen Ausgangssituation werden benutzerbezogen gespeichert.
F05	Ziele	Finanzielle Ziele können angelegt, angezeigt, geändert und entfernt werden.
F06	Lebensereignisse	Planbare Ereignisse werden zeitlich und finanziell erfasst.
F07	Passwort zurücksetzen	Ein Reset-Link wird per E-Mail versandt und ermöglicht die Vergabe eines neuen Passwortes.
F08	Prognosen	Gespeicherte Werte können für finanzielle Projektionen und Szenarien verwendet werden.
F09	FAQ	Häufige Fragen werden thematisch gegliedert und über die Navigation erreichbar gemacht.
F10	Assistenzfunktion	Ein Finanzassistent kann als ergänzende Hilfestellung eingebunden werden.

3.3 Nichtfunktionale Anforderungen

Neben den sichtbaren Funktionen beeinflussen nichtfunktionale Anforderungen maßgeblich die Qualität des Systems. Für den Prototyp wurde insbesondere Wartbarkeit, Nachvollziehbarkeit, Sicherheit und Portabilität berücksichtigt.

Qualitätsmerkmal	Umsetzung im Projekt
Benutzbarkeit	Klare Navigation, verständliche Beschriftung und konsistente Ansprache der Nutzenden.
Wartbarkeit	Trennung in Frontend, Routen, Services und Datenbankzugriff.
Sicherheit	Passwort-Hashing, serverseitige Sitzungen, gehashte Reset-Tokens und Umgebungsvariablen.
Portabilität	Containerisierung von Frontend und Backend mit Docker Compose.
Erweiterbarkeit	Blueprints und separate Services ermöglichen zusätzliche Endpunkte und Funktionen.
Datenkonsistenz	Relationale Tabellen und eindeutige Zuordnung zu Nutzerkonten.

3.4 Anwendungsfälle

Ein typischer Ablauf beginnt mit der Registrierung. Nach erfolgreicher Anmeldung erfasst die Person ihre finanzielle Ausgangssituation. Dazu gehören je nach Oberfläche beispielsweise Einkommen, regelmäßige Ausgaben, Rücklagen, Vermögenswerte oder bestehende Verpflichtungen. Danach können Ziele und erwartete Lebensereignisse ergänzt werden. Die gespeicherten Daten bilden die Grundlage für Auswertungen und Prognosen. Bei einem vergessenen Passwort fordert die Person einen Reset-Link an, öffnet diesen aus der E-Mail und vergibt innerhalb des Gültigkeitszeitraums ein neues Passwort.

4 Technologische Grundlage

4.1 React und Vite im Frontend

Das Frontend basiert auf React. React ermöglicht den Aufbau der Oberfläche aus wiederverwendbaren Komponenten und unterstützt eine zustandsbasierte Darstellung. Änderungen an Eingabewerten oder geladenen Daten können gezielt in betroffenen Komponenten aktualisiert werden. Vite dient als Entwicklungs- und Build-Werkzeug. Der Entwicklungsserver wird standardmäßig über Port 5173 bereitgestellt. Diese Kombination eignet sich für den Prototyp, da sie

einen schnellen Entwicklungszyklus und eine klare Strukturierung der Benutzeroberfläche ermöglicht.

Die Benutzeroberfläche übernimmt die Eingabevalidierung auf Darstellungsebene, die Navigation zwischen den Seiten und die Kommunikation mit dem Backend. Sicherheitsrelevante Prüfungen dürfen jedoch nicht ausschließlich im Frontend stattfinden, da Anfragen auch direkt an die API gesendet werden können. Deshalb validiert das Flask-Backend wesentliche Eingaben erneut.

4.2 Flask im Backend

Flask ist ein leichtgewichtiges Python-Webframework. Im Projekt wird die Anwendung in mehrere Blueprints aufgeteilt. Vorhanden sind unter anderem Routen für Authentifizierung, Nutzerdaten, Finanzprofile, Prognosen und Chatfunktionen. Diese Trennung reduziert die Größe einzelner Dateien und ermöglicht eine fachliche Gruppierung der Endpunkte. Die zentrale Anwendungsdatei registriert die Blueprints, konfiguriert CORS und initialisiert benötigte Erweiterungen.

4.3 PostgreSQL und Supabase

PostgreSQL ist ein relationales Datenbankmanagementsystem. Supabase stellt unter anderem eine gehostete PostgreSQL-Datenbank und eine Schnittstelle für den Datenzugriff bereit. Gegenüber einer ausschließlich dokumentenorientierten Speicherung bietet das relationale Modell klare Beziehungen, Datentypen und Integritätsbedingungen. Finanzprofile, Ziele und Lebensereignisse können dadurch getrennt gespeichert und über Fremdschlüssel verbunden werden.

Der Wechsel zu PostgreSQL erfordert eine Überarbeitung des Datenzugriffs. Funktionen, die zuvor vollständige JSON-Dokumente gelesen oder geschrieben hatten, mussten auf Tabellenoperationen abgebildet werden. Gleichzeitig war zu berücksichtigen, dass ein Finanzprofil aus mehreren fachlich zusammengehörenden Datensätzen bestehen kann.

4.4 Docker und Docker Compose

Docker kapselt die benötigte Laufzeitumgebung. Im Projekt werden Frontend und Backend als getrennte Services definiert. Das Backend wird auf Port 5000, das Frontend auf Port 5173 veröffentlicht. Docker Compose legt Abhängigkeiten, Volumes, Umgebungsvariablen und Build-Kontexte fest. Dadurch kann das System mit einem zentralen Befehl gestartet werden und verhält sich auf unterschiedlichen Entwicklungsrechnern weitgehend konsistent.

4.5 Git und GitHub

Git wurde zur Versionsverwaltung eingesetzt, GitHub als zentrales Repository. Die Commit-Historie zeigt unter anderem Erweiterungen für die Datenbankmigration, FAQ-Seite, Chatfunktion,

Bearbeitung von Zielen, Dokumentation und Tests. Versionsverwaltung ist für das Teamprojekt besonders relevant, da Änderungen nachvollziehbar bleiben und parallele Arbeit an getrennten Aufgabenbereichen möglich ist.

5 Systemarchitektur

5.1 Architekturübersicht

Das System folgt einer dreischichtigen Client-Server-Architektur. Die Präsentationsschicht wird durch das React-Frontend gebildet. Die Anwendungsschicht besteht aus Flask-Routen und fachlichen Services. Die Persistenzschicht wird durch PostgreSQL beziehungsweise Supabase bereitgestellt. Zwischen Frontend und Backend werden HTTP-Anfragen ausgetauscht. Daten werden überwiegend im JSON-Format übertragen.

5.2 Frontend-Schicht

Die Frontend-Schicht stellt Formulare, Navigation, Übersichten und Szenario Seiten bereit. Sie sammelt Eingaben und sendet diese an die API. Bei erfolgreichen Antworten aktualisiert sie den sichtbaren Zustand; bei Fehlern werden verständliche Rückmeldungen ausgegeben. Für geschützte Bereiche muss die Sitzung des Nutzers berücksichtigt werden. Da das Backend Cookies für die Authentifizierung verwendet, werden Anfragen mit Zugangsdaten beziehungsweise Cookies übertragen.

5.3 Backend-Schicht

Die Backend-Schicht verarbeitet HTTP-Anfragen, prüft Eingaben und koordiniert den Datenbankzugriff. Die Authentifizierungsrouten implementieren Registrierung, Login, Logout sowie Passwort-Wiederherstellung. Finanzprofilrouten verwalten die finanziellen Daten. Weitere Blueprints kapseln Nutzer-, Prognose- und Chatfunktionen. Der eigentliche Zugriff auf die Datenbank wird in Services ausgelagert. Dadurch können Routen kürzer gehalten und Datenbankdetails zentral angepasst werden.

5.4 Kommunikation und CORS

Frontend und Backend laufen während der Entwicklung auf unterschiedlichen Ports. Browser behandeln diese Kombination als unterschiedliche Ursprünge. Deshalb konfiguriert das Backend CORS für die lokalen Frontend-Adressen und optional für eine über Umgebungsvariablen definierte produktive URL. Die Option zur Übertragung von Zugangsdaten ist aktiviert, damit der Browser das Sitzungs-Cookie bei API-Anfragen mitsenden kann.

6 Datenbankkonzept und Migration

6.1 Entwicklung der Datenbankstrategie

Die Datenbankstrategie wurde im Projektverlauf mehrfach angepasst. Ursprünglich war Azure Cosmos DB vorgesehen. Dieser Ansatz begünstigt die Speicherung umfangreicher Objekte als JSON-Dokumente. Ein Finanzprofil kann dabei gemeinsam mit eingebetteten Zielen und Ereignissen in einem Dokument abgelegt werden. Nach dem Wechsel auf Supabase und PostgreSQL musste dieses Modell neu bewertet werden. PostgreSQL unterstützt zwar JSON-Spalten, seine Stärken liegen jedoch in strukturierten Tabellen, Beziehungen und Abfragen über eindeutig typisierte Attribute.

Als Zwischenlösung konnten vorhandene JSON-Daten weiterhin in einem Profelfeld gespeichert werden. Für eine nachhaltigere Struktur wurden Ziele und Lebensereignisse anschließend in Kind Tabellen ausgelagert. Das Repository enthält hierfür ein Migrationsskript, das die entsprechenden Arrays aus bestehenden Profilen liest und in relationale Tabellen überführt.

6.2 Relationales Zielmodell

Das genaue Schema kann im weiteren Projektverlauf erweitert werden. Fachlich bietet sich folgende Struktur an: Ein Nutzer besitzt höchstens ein zentrales Finanzprofil. Ein Finanzprofil kann mehrere finanzielle Ziele und mehrere Lebensereignisse enthalten. Die Kind Tabellen referenzieren das Profil oder direkt den Nutzer über Fremdschlüssel.

6.3 Überführung von JSON in Tabellen

Bei der Migration werden zunächst vorhandene Profildatensätze gelesen. Enthält ein Profil eingebettete Arrays für Ziele oder Lebensereignisse, wird jedes Element einzeln in die vorgesehene Kind Tabelle geschrieben. Entscheidend ist, dass die Zuordnung zum ursprünglichen Profil erhalten bleibt. Nach erfolgreicher Migration können die JSON-Felder bereinigt oder nur noch für Abwärtskompatibilität vorübergehend beibehalten werden.

- Tabellen und Beziehungen in PostgreSQL anlegen.
- Vorhandene Profile mit JSON-Inhalten auslesen.
- Ziele und Lebensereignisse validieren und normalisieren.
- Datensätze in die jeweiligen Kind Tabellen einfügen.
- Anzahl und Zuordnung der migrierten Datensätze kontrollieren.
- Alte JSON-Strukturen erst nach erfolgreicher Prüfung entfernen.

6.4 Vorteile und Risiken der Migration

Die relationale Struktur verbessert Abfragbarkeit und Konsistenz. Einzelne Ziele können verändert oder gelöscht werden, ohne das gesamte Finanzprofil zu überschreiben. Gleichzeitig steigt die Komplexität des Schreibvorgangs, weil mehrere Tabellen beteiligt sein können. Unvollständige

Migrationen oder fehlgeschlagene Teiloperationen stellen ein Risiko dar. Für eine produktive Weiterentwicklung sollten solche Vorgänge in Datenbanktransaktionen ausgeführt werden. Zusätzlich sind eindeutige Constraints und Fremdschlüssel mit sinnvoller Löschrategie erforderlich.

6.5 Datenschutz und Datenminimierung

Finanzdaten zählen zu besonders sensiblen personenbezogenen Informationen. Daher sollten nur Daten gespeichert werden, die für die Funktionen tatsächlich erforderlich sind. Zugangsdaten, Datenbank-URLs und E-Mail-Zugangsdaten gehören nicht in das Repository, sondern in Umgebungsvariablen. Für eine reale Produktivnutzung wären außerdem ein Löschkonzept, Auskunftsfunktionen, Protokollierung sicherheitsrelevanter Vorgänge und eine nachvollziehbare Einwilligungs- beziehungsweise Datenschutzerklärung notwendig.

7 Implementierung zentraler Funktionen

7.1 Registrierung

Der Registrierungsendpoint akzeptiert Vorname, Nachname, Geburtsdatum, E-Mail-Adresse und Passwort. Zunächst werden Zeichenketten bereinigt und die E-Mail-Adresse normalisiert. Fehlende Pflichtfelder führen zu einer Antwort mit dem HTTP-Status 400. Das Passwort muss mindestens sechs Zeichen enthalten. Vor dem Anlegen wird geprüft, ob die E-Mail-Adresse bereits existiert. In diesem Fall wird der Status 409 zurückgegeben. Bei erfolgreicher Erstellung antwortet das Backend mit Status 201.

Die Registrierung zeigt eine klare Trennung zwischen HTTP-Verarbeitung und Datenzugriff. Die Route validiert die Anfrage, während der User-Service den eigentlichen Datensatz erzeugt und das Passwort gehasht speichert. Für eine produktive Version wäre eine strengere Passwortregel sinnvoll, beispielsweise eine höhere Mindestlänge und eine Prüfung kompromittierter Passwörter.

7.2 Login und Sitzung

Beim Login werden E-Mail-Adresse und Passwort entgegengenommen. Das Backend sucht den Nutzer über die normalisierte E-Mail-Adresse. Der gespeicherte Passwort-Hash wird mit dem eingegebenen Passwort verglichen. Bei ungültigen Zugangsdaten antwortet die API bewusst mit einer einheitlichen Fehlermeldung. Nach erfolgreicher Prüfung werden Nutzer-ID, E-Mail-Adresse, Vorname, Nachname und Geburtsdatum in der Flask-Sitzung gespeichert. Die Sitzung dient anschließend zur Zuordnung geschützter Anfragen.

Der Logout leert die serverseitig verwalteten Sitzungsinformationen. Aus Sicht des Frontends muss danach auch der lokale Authentifizierungszustand zurückgesetzt und auf eine öffentliche Seite navigiert werden.

7.3 Finanzprofil

Für die Eingaben zur finanziellen Situation wurden ein `financial_profile_service` und zugehörige `financial_profile_routes` erstellt. Die Routen nehmen die vom Frontend übermittelten Werte entgegen und ordnen sie dem angemeldeten Nutzer zu. Der Service kapselt die notwendigen Datenbankoperationen. Dadurch können Änderungen an Tabellen oder am Datenbankanbieter vorgenommen werden, ohne jede Route vollständig neu zu implementieren.

Bei der Verarbeitung ist zwischen erstmaligem Anlegen und späterem Aktualisieren zu unterscheiden. Existiert noch kein Profil, wird ein neuer Datensatz erzeugt. Ist bereits ein Profil vorhanden, werden definierte Felder aktualisiert. Unbekannte oder nicht erlaubte Attribute sollten ignoriert oder zurückgewiesen werden. Diese Whitelist-Strategie verhindert, dass beliebige Spalten über manipulierte Anfragen verändert werden.

7.4 Ziele und Lebensereignisse

Ziele und Lebensereignisse sind für längerfristige Szenarien relevant. Ein Ziel kann beispielsweise aus Bezeichnung, Zielbetrag und Zieldatum bestehen. Lebensereignisse bilden erwartete finanzielle Veränderungen ab. Die relationale Speicherung ermöglicht CRUD-Operationen auf einzelnen Elementen. Die Commit-Historie dokumentiert unter anderem Endpunkte zum Bearbeiten und Löschen von Zielen. Das Frontend kann damit Datensätze gezielt aktualisieren, ohne das vollständige Profil neu zu übertragen.

7.5 Passwort vergessen Funktion

Die Passwort-Wiederherstellung besteht aus zwei Endpunkten. Beim ersten Endpunkt übermittelt die Person ihre E-Mail-Adresse. Ist ein passendes Konto vorhanden, erzeugt das Backend mit einer kryptografisch geeigneten Zufallsfunktion ein Token. Dieses Token wird nicht im Klartext gespeichert. Stattdessen wird ein SHA-256-Hash abgelegt und mit einer Gültigkeit von 30 Minuten versehen. Der Klartext-Token erscheint nur im Link, der per E-Mail versandt wird.

Unabhängig davon, ob die E-Mail-Adresse existiert, liefert der Endpunkt dieselbe Erfolgsmeldung. Dadurch können Angreifer nicht anhand der Antwort prüfen, welche E-Mail-Adressen registriert sind. Beim zweiten Endpunkt werden Token und neues Passwort übermittelt. Das Backend hasht den empfangenen Token, sucht den zugehörigen Datensatz, kontrolliert den Ablaufzeitpunkt und aktualisiert anschließend das Passwort. Ein ungültiger oder abgelaufener Link wird zurückgewiesen.

7.6 E-Mail-Service

Für den Reset-Prozess wurde ein E-Mail-Server beziehungsweise E-Mail-Service konfiguriert. Der Service erhält die Empfängeradresse und die erzeugte Reset-URL. Zugangsdaten und Absenderkonfiguration werden über Umgebungsvariablen bereitgestellt. Dadurch bleiben

vertrauliche Informationen außerhalb des Quellcodes. Bei Versandfehlern sollte die Anwendung den Fehler serverseitig protokollieren, ohne interne Konfigurationsdetails an das Frontend weiterzugeben.

7.7 FAQ-Seite

Die FAQ-Seite bündelt häufige Fragen und erste Hilfestellungen. Sie wurde in die Navigation eingebunden und später nach Themen gruppiert. Eine gruppierte Darstellung verbessert die Auffindbarkeit, weil Fragen beispielsweise nach Konto, Finanzprofil, Prognosen und Datenschutz gegliedert werden können. Die FAQ reduziert wiederkehrende Verständnisprobleme und ergänzt die eigentliche Funktionalität um eine niedrighschwellige Erklärungsebene.

7.8 Prognosen und Finanzassistent

Das Backend registriert getrennte Routen für Prognosen und Chatfunktionen. Prognosen können auf gespeicherten Profilwerten aufbauen und Entwicklungen über einen definierten Zeitraum berechnen. Der Finanzassistent dient als ergänzende Interaktionsmöglichkeit. Bei einer KI-gestützten Funktion ist besonders wichtig, dass Antworten als Unterstützung und nicht als verbindliche Finanzberatung dargestellt werden. Sensible Profildaten sollten nur in dem Umfang an externe Modelle übertragen werden, der technisch und datenschutzrechtlich erforderlich ist.

8 Sicherheitskonzept

8.1 Passwortspeicherung

Passwörter dürfen nicht im Klartext gespeichert werden. Das Projekt verwendet bcrypt zum Erzeugen und Prüfen von Passwort-Hashes. Bcrypt integriert einen Salt und ist absichtlich rechenintensiv, wodurch massenhafte Passwortversuche gegen gestohlene Hashes erschwert werden. Der Klartext wird nur während der Verarbeitung der aktuellen Anfrage benötigt und nicht dauerhaft gespeichert.

8.2 Sitzungs- und Cookie-Konfiguration

Flask verwendet einen geheimen Schlüssel zur Absicherung der Sitzung. Dieser Schlüssel wird aus einer Umgebungsvariablen geladen; fehlt die Variable, erzeugt die Anwendung für die laufende Instanz einen zufälligen Wert. Für Cookies sind SameSite=None und Secure aktiviert. Dadurch kann das Cookie bei getrennten Frontend- und Backend-Ursprüngen übertragen werden, wird jedoch nur über HTTPS gesendet. Für lokale Entwicklung kann diese Einstellung je nach Browser und Protokoll zusätzliche Konfiguration erfordern.

Für eine produktive Bereitstellung sollten außerdem HttpOnly, eine angemessene Sitzungsdauer, Schutz gegen Cross-Site-Request-Forgery und eine zuverlässige serverseitige Sitzungsablage

geprüft werden. Ein Neustart mit zufällig erzeugtem Secret Key würde bestehende Sitzungen ungültig machen; produktiv ist daher ein dauerhaft gesetzter Schlüssel erforderlich.

8.3 Schutz des Passwort-Reset-Prozesses

- Kryptografisch zufälliger Token
- Nur der Hash wird in der Datenbank gespeichert
- Begrenzte Gültigkeit von 30 Minuten
- Einheitliche Antwort unabhängig von der Existenz eines Kontos
- Erneutes Hashing und Vergleich beim Zurücksetzen

Zusätzlich sollte ein Token nach erfolgreicher Verwendung sofort gelöscht oder ungültig gemacht werden. Rate Limiting kann verhindern, dass der Versand massenhaft ausgelöst wird. Ebenso sollte die Anwendung vermeiden, vollständige Reset-Links in öffentlich zugängliche Protokolle zu schreiben.

8.4 Eingabevalidierung und Fehlerbehandlung

Das Backend prüft Pflichtfelder und grundlegende Passwortbedingungen. Datenbankseitige Constraints bilden eine zweite Schutzebene. Für weitere Eingabefelder sollten Datentypen, Wertebereiche und Datumsangaben kontrolliert werden. Fehlerantworten müssen verständlich sein, dürfen jedoch keine Stacktraces, Datenbankdetails oder Zugangsdaten offenlegen. Zentralisiertes Logging unterstützt die Fehlersuche, ohne interne Informationen an den Client zu senden.

8.5 Grenzen des aktuellen Sicherheitsstands

Der Prototyp enthält wesentliche Grundmaßnahmen, ersetzt jedoch kein vollständiges Security Review. Vor einem produktiven Einsatz wären automatisierte Abhängigkeitsprüfungen, sichere HTTP-Header, CSRF-Schutz, Rate Limiting, Berechtigungstests, Datenschutzprüfung und ein Secret-Management erforderlich. Besonders bei Finanzdaten muss außerdem überprüft werden, dass jeder Endpunkt ausschließlich auf Datensätze des angemeldeten Nutzers zugreifen kann.

9 Bereitstellung und Inbetriebnahme

9.1 Voraussetzungen

Für den vorgesehenen Standardweg wird Docker Desktop benötigt. Zusätzlich müssen die erforderlichen Zugangsdaten für Supabase, der Flask-Schlüssel, die Frontend-Basis-URL und gegebenenfalls die E-Mail-Konfiguration vorhanden sein. Das Repository enthält eine Beispieldatei für Umgebungsvariablen, die als lokale .env-Datei übernommen und ausgefüllt wird. Die produktive .env-Datei darf nicht versioniert werden.

9.2 Start mit Docker

- Repository klonen oder als Archiv herunterladen.
- Example.env nach .env kopieren.
- Supabase-, Flask- und E-Mail-Werte ergänzen.
- Docker Desktop starten.
- Im Projektverzeichnis docker compose up -build ausführen.
- Frontend unter <http://localhost:5173> öffnen.

Beim ersten Start werden die Images gebaut und Abhängigkeiten installiert. Spätere Starts können in der Regel mit docker compose up erfolgen. Durch gemountete Volumes werden Änderungen während der Entwicklung direkt in die Container übernommen.

9.3 Start ohne Docker

Alternativ lassen sich Backend und Frontend in getrennten Terminals starten. Für das Backend werden die Python-Abhängigkeiten installiert und app.py ausgeführt. Im Frontend werden die Node-Abhängigkeiten installiert und der Vite-Entwicklungsserver gestartet.

10 Tests und Qualitätssicherung

10.1 Teststrategie

Im Zuge der Demo-Vorbereitung wurde die Website manuell getestet. Zusätzlich enthält die aktuelle Commit-Historie Hinweise auf Unit-Tests für ausgewählte Routen und manuelle Tests für Berechnungsfunktionen. Für den Prototyp ist eine Kombination aus manuellen Oberflächentests, API-Tests und gezielten Unit-Tests zweckmäßig. Entscheidend ist, sowohl erfolgreiche Abläufe als auch Fehlerfälle abzudecken.

10.2 Manuelle Tests

Testfall	Erwartetes Ergebnis
Registrierung mit vollständigen Daten	Konto wird angelegt; Erfolgsmeldung erscheint.
Registrierung mit bestehender E-Mail	Anfrage wird mit verständlicher Fehlermeldung abgewiesen.
Login mit korrekten Daten	Sitzung wird erstellt; geschützter Bereich ist erreichbar.
Finanzprofil speichern	Werte bleiben nach erneutem Laden erhalten.

Ziel bearbeiten und löschen	Nur der ausgewählte Datensatz wird geändert beziehungsweise entfernt.
Passwort-Reste anfordern	Neutrale Bestätigung; bei vorhandenem Konto wird E-Mail versandt.
Abgelaufenen Reset-Link verwenden	Zurücksetzen wird abgewiesen.
FAQ-Navigation	FAQ-Seite wird korrekt geöffnet und Gruppen werden angezeigt.
Logout	Sitzung endet; geschützte Inhalte sind nicht mehr zugänglich.

10.3 Automatisierte Tests

Automatisierte Tests sollten Routen isoliert gegen eine Testkonfiguration ausführen. Datenbank- und E-Mail-Zugriffe können gemockt werden, damit die Tests reproduzierbar bleiben. Für die Authentifizierungsrouten sind insbesondere Statuscodes, Antworttexte, Session-Zustand und Service-Aufrufe zu prüfen. Für Finanzberechnungen sollten bekannte Eingabewerte mit erwarteten Ergebnissen verglichen und Grenzfälle wie null Prozent Rendite, negative Werte oder sehr lange Laufzeiten berücksichtigt werden.

10.4 Qualitätskriterien

- Alle zentralen Nutzerflüsse sind ohne kritische Fehler ausführbar.
- Fehlerfälle erzeugen definierte Startcodes und verständliche Meldungen.
- Daten bleiben konsistent einem Nutzer zugeordnet.
- Die Anwendung ist über die dokumentierten Schritte reproduzierbar startbar.
- Sicherheitsrelevante Geheimnisse befinden sich nicht im Repository.
- Änderungen an Zielen und Profilen führen nicht zu unbeabsichtigtem Datenverlust.

10.5 Festgestellte Grenzen

Die Dokumentation der vorhandenen Tests im Repository ist noch ausbaufähig. Für eine belastbare Freigabe sollten Testabdeckung und Testergebnisse automatisiert in einer CI-Pipeline ausgewiesen werden. Ebenso fehlen für einen produktiven Einsatz Lasttests, browserübergreifende Tests, Barrierefreiheitstests und eine systematische Sicherheitsprüfung. Diese Punkte schmälern nicht den Prototypcharakter, markieren aber wichtige nächste Schritte.

11 Projektorganisation und Herausforderungen

11.1 Aufgabenverteilung

Das Projekt wurde arbeitsteilig umgesetzt. Ein Teammitglied übernahm schwerpunktmäßig das Backend für Registrierung und Login, die Anpassung der Datenbankzugriffe, die Speicherung des Finanzprofils, die Einrichtung des E-Mail-Servers und die Passwort-Wiederherstellung. Zusätzlich wurden die Website im Rahmen der Demo-Vorbereitung getestet sowie die FAQ-Seite und ihre Navigation umgesetzt. Nach dem Wechsel zu PostgreSQL wurde das Datenmodell von JSON-Strukturen in relationale Tabellen überführt.

Weitere Aufgabenbereiche des Teams umfassten unter anderem Frontend-Komponenten, finanzielle Szenarien, Prognosefunktionen, Zielverwaltung, Chat- beziehungsweise Assistenzfunktionen, UI-Anpassungen und Projektdokumentation. Eine klare fachliche Aufteilung erleichtert die Parallelisierung, erfordert jedoch abgestimmte Schnittstellen zwischen Frontend, Backend und Datenbank.

11.2 Wechsel der Datenbanktechnologie

Die größte technische Herausforderung war der Wechsel von Azure Cosmos DB zu Supabase/PostgreSQL. Dabei änderte sich nicht nur der Verbindungsaufbau, sondern auch das grundlegende Datenmodell. Dokumentenorientierte Daten konnten zunächst als JSON übernommen werden, waren für gezielte Bearbeitung einzelner Unterelemente jedoch weniger geeignet. Die spätere Normalisierung in relationale Tabellen erforderte Anpassungen an Schema, Services, Routen und Migrationslogik. Diese Entwicklung zeigt, dass Infrastrukturentscheidungen möglichst früh anhand fachlicher Abfragen und Änderungsanforderungen bewertet werden sollten.

11.3 Schnittstellenabstimmung

Frontend und Backend müssen sich auf Endpunkte, HTTP-Methoden, Feldnamen, Datentypen und Fehlerantworten einigen. Änderungen am Datenmodell wirken sich daher häufig über mehrere Schichten aus. Eine schriftliche API-Spezifikation oder OpenAPI-Dokumentation hätte die Abstimmung zusätzlich erleichtert. Für kleine Teams kann bereits eine zentrale Tabelle mit Route, Methode, Request, Response und Zuständigkeit Missverständnisse deutlich reduzieren.

11.4 Versionsverwaltung

Die Git-Historie dokumentiert die schrittweise Erweiterung des Systems. Aussagekräftige Commit-Nachrichten sind jedoch nicht durchgängig vorhanden. Für künftige Projekte empfiehlt sich ein einheitliches Schema, beispielsweise Conventional Commits. Feature-Banches und Pull Requests ermöglichen Reviews, bevor Änderungen in den Hauptbranch gelangen. Bei Datenbankänderungen sollten Migrationsdateien gemeinsam mit dem zugehörigen Anwendungscode versioniert werden.

11.5 Erkenntnisse aus dem Projekt

- Eine saubere Trennung von Routen und Services erleichtert Technologiewechsel.
- Datenmodelle sollten aus konkreten Anwendungsfällen und Abfragen abgeleitet werden.
- Sicherheitsfunktionen benötigen neben dem sichtbaren Ablauf zusätzliche Missbrauchsschutzmaßnahmen.
- Containerisierung verbessert die Reproduzierbarkeit, ersetzt aber keine vollständige Deploymentstrategie.
- Frühe End-to-End-Tests reduzieren Integrationsprobleme kurz vor einer Demo.

12 Bewertung und Ausblick

12.1 Bewertung der Zielerreichung

Die zentralen Ziele des Prototyps wurden erreicht. Die Anwendung besitzt eine moderne Frontend-Backend-Struktur, ermöglicht Benutzerverwaltung und verarbeitet individuelle Finanzdaten. Die Passwort-Wiederherstellung wurde inklusive E-Mail-Versand umgesetzt. Docker unterstützt einen reproduzierbaren Start. Die Datenbankmigration zeigt, dass das System an geänderte technische Rahmenbedingungen angepasst werden konnte.

Besonders positiv ist die Modularisierung des Backends. Die Blueprints bilden fachliche Bereiche ab, während Services den Datenbankzugriff kapseln. Dadurch bleibt die Anwendung trotz wachsendem Funktionsumfang überschaubar. Gleichzeitig bestehen typische Prototypgrenzen bei Testabdeckung, Security Hardening, Fehlerbehandlung und produktiver Betriebsfähigkeit.

12.2 Technische Weiterentwicklungen

- OpenAPI-Spezifikation und automatisch generierte API-Dokumentationen
- Vollständige CI-Pipeline mit Backend-, Frontend- und Integrationstests
- Datenbankmigrationen über ein Werkzeug wie Alembic
- Transaktionen für zusammengesetzte Profiländerungen
- Rate Limiting und CSRF-Schutz
- Zentrales Monitoring und strukturiertes Logging
- Produktionsfähige Container und Reverse Proxy mit HTTPS
- Barrierefreie Formulare und Tastaturnavigation

12.3 Fachliche Weiterentwicklungen

- Berücksichtigung von Inflation, Steuern und unterschiedlichen Renditenszenarien
- Vergleich mehrerer Lebens- und Sparszenarien
- Export von Profilen und Prognosen als PDF oder CSV

- Erinnerungen für Ziele und geplante Ereignisse
- Visualisierung von Unsicherheiten und Sensitivitätsanalysen
- Feingranulare Erklärungen zu Berechnungen und Annahmen
- Optionaler Import aus Bankexporten ohne direkte Kontoverbindung

12.4 Risiken bei einer Produktivsetzung

Vor einem realen Betrieb müssten rechtliche, datenschutzbezogene und sicherheitstechnische Anforderungen vertieft geprüft werden. Finanzdaten erfordern eine hohe Schutzklasse. Dazu gehören Verschlüsselung, Zugriffskontrolle, Backup- und Wiederherstellungstests, Auftragsverarbeitungsverträge, Löschfristen sowie eine klare Abgrenzung von Finanzberatung. Bei einer KI-Komponente müssen zusätzlich Datenweitergabe, Modellantworten und mögliche Fehlberatung berücksichtigt werden.

13 Fazit

Der Lebens-Finanzplaner demonstriert die Entwicklung einer vollständigen webbasierten Anwendung über mehrere technischen Schichten hinweg. React ermöglicht eine komponentenbasierte Benutzeroberfläche, Flask stellt die API und die fachliche Verarbeitung bereit, PostgreSQL sorgt für eine strukturierte Datenhaltung. Docker verbindet die Komponenten zu einer reproduzierbaren Entwicklungsumgebung.

Der Wechsel von Azure Cosmos DB beziehungsweise JSON-orientierter Speicherung zu Supabase/PostgreSQL war zugleich die größte Herausforderung und eine zentrale Lernerfahrung. Die Überführung in relationale Tabellen verbesserte die gezielte Bearbeitung und Konsistenz von Finanzziele und Lebensereignissen. Durch die Trennung in Routen und Services konnten notwendige Anpassungen systematisch umgesetzt werden.

Mit Registrierung, Login, Finanzprofil, Passwort-Wiederherstellung, FAQ, Prognose- und Assistenzfunktionen deckt der Prototyp einen umfangreichen Funktionsbereich ab. Für eine produktive Nutzung sind weitere Tests, Sicherheitsmaßnahmen und rechtliche Prüfungen erforderlich. Als Hochschulprojekt zeigt die Anwendung jedoch nachvollziehbar, wie Frontend, Backend, Datenbank, E-Mail-Service und Containerisierung zu einem kohärenten Gesamtsystem integriert werden können.